

Jetpack Compose第二章第一节教学讲义（布局与列表核心）

各位同学，上节课我们掌握了Compose的核心思想，这节课咱们聚焦“UI骨架搭建”的核心技能——标准布局组件、Slots API设计模式和列表组件。这三部分是任何Compose页面都绕不开的基础，学会它们就能轻松搭建出结构清晰、性能优异的APP界面。全程用“组件功能+代码实操+效果说明”的方式，保证大家学完就能用！

一、标准布局组件：Compose的“基础货架”

布局组件的作用就像“货架”，决定了UI元素的摆放位置和排列方式。Compose提供了3个最核心的标准布局组件，覆盖90%以上的布局场景，它们分别对应传统布局的LinearLayout（垂直/水平）和FrameLayout。

1. Column：垂直排列的“竖货架”

功能：让子组件从上到下垂直排列，类似传统的 `LinearLayout(orientation=vertical)`，是页面纵向布局的首选。

基础用法：简单垂直排列

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.material3.Button
3  import androidx.compose.material3.Text
4  import androidx.compose.foundation.layout.padding
5  import androidx.compose.ui.unit.dp
6  import androidx.compose.ui.tooling.preview.Preview
7
8  // 垂直排列的按钮和文本
9  @Composable
10 fun SimpleColumn() {
11     Column(
12         // 内边距：给整个Column周围留16dp空间
13         modifier = Modifier.padding(16.dp)
14     ) {
15         Text(text = "请完成以下操作", modifier = Modifier.padding(bottom = 8.dp))
16         Button(onClick = { /* 点击逻辑 */ }, modifier = Modifier.padding(bottom
17             = 8.dp)) {
18             Text("第一步：点击我")
19         }
20     }
21 }
```

```

18         }
19         Button(onClick = { /* 点击逻辑 */ }) {
20             Text("第二步：再点击我")
21         }
22     }
23 }
24
25 // 预览效果
26 @Preview(showBackground = true)
27 @Composable
28 fun SimpleColumnPreview() {
29     SimpleColumn()
30 }

```

进阶用法：控制对齐和间距

通过 `horizontalAlignment` 控制子组件的水平对齐方式，通过 `verticalArrangement` 控制子组件的垂直间距。

Code block

```

1  import androidx.compose.foundation.layout.Arrangement
2
3  @Composable
4  fun AdvancedColumn() {
5      Column(
6          modifier = Modifier
7              .padding(16.dp)
8              .fillMaxWidth(), // 占满父容器宽度
9          horizontalAlignment = Arrangement.Center, // 子组件水平居中
10         verticalArrangement = Arrangement.SpaceEvenly // 子组件垂直间距均匀分布
11     ) {
12         Text(text = "居中的标题")
13         Button(onClick = {}) { Text("居中按钮1") }
14         Button(onClick = {}) { Text("居中按钮2") }
15     }
16 }
17
18 @Preview(showBackground = true)
19 @Composable
20 fun AdvancedColumnPreview() {
21     AdvancedColumn()
22 }

```

2. Row：水平排列的“横货架”

功能：让子组件从左到右水平排列，类似传统的

`LinearLayout(orientation=horizontal)`，常用于一行内的元素布局（如标题栏、列表项）。

实战场景：标题栏布局

Code block

```
1  import androidx.compose.foundation.layout.Row
2  import androidx.compose.foundation.layout.Spacer
3  import androidx.compose.foundation.layout.fillMaxWidth
4  import androidx.compose.foundation.layout.weight
5
6  @Composable
7  fun TitleBar() {
8      Row(
9          modifier = Modifier
10             .fillMaxWidth()
11             .padding(16.dp),
12          horizontalArrangement = Arrangement.SpaceBetween, // 子组件左右对齐
13          verticalAlignment = androidx.compose.ui.Alignment.CenterVertically //
            垂直居中
14      ) {
15          // 左侧返回按钮
16          Button(onClick = {}, modifier = Modifier.weight(1f)) { // weight占比1
17              Text("返回")
18          }
19          // 中间标题 (用Spacer填充空白, 实现标题居中)
20          Text(text = "我的页面", modifier = Modifier.weight(2f)) // weight占比2
21          // 右侧设置按钮
22          Button(onClick = {}, modifier = Modifier.weight(1f)) {
23              Text("设置")
24          }
25      }
26  }
27
28  @Preview(showBackground = true)
29  @Composable
30  fun TitleBarPreview() {
31      TitleBar()
32  }
```

 关键知识点：`weight` 属性用于分配水平空间占比，总和为4时，左侧和右侧各占1份，中间标题占2份，实现标题栏的经典布局。

3. Box：层叠对齐的“展示台”

功能：子组件可以层叠排列，也能控制子组件在容器内的对齐方式（如居中、居右等），类似传统的FrameLayout，常用于“背景+前景”的层叠场景。

实战场景：带背景的文本

Code block

```
1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.layout.Box
3  import androidx.compose.foundation.layout.size
4  import androidx.compose.ui.Alignment
5  import androidx.compose.ui.graphics.Color
6  import androidx.compose.ui.unit.dp
7
8  @Composable
9  fun BoxWithBackground() {
10     Box(
11         modifier = Modifier
12             .size(200.dp) // 宽高各200dp
13             .background(Color.LightGray), // 灰色背景
14         contentAlignment = Alignment.Center // 子组件居中
15     ) {
16         // 前景文本（层叠在背景之上）
17         Text(text = "层叠的文本", color = Color.Black)
18     }
19 }
20
21 @Preview(showBackground = true)
22 @Composable
23 fun BoxWithBackgroundPreview() {
24     BoxWithBackground()
25 }
```

二、Slots API：Compose的“组件组装魔法”

很多同学在用Compose组件时会疑惑：“为什么Button里面能放Text，Card里面能放Column？”这背后就是Slots API的功劳——它是Compose组件的核心设计模式，让组件像“乐高积木”一样可灵活组装。

1. 什么是Slots API?

Slots（插槽）就是组件预留的“内容入口”——组件开发者在定义组件时，用函数参数预留出放置子内容的位置，使用者在调用组件时，把自己的内容“插入”到这些插槽中。

类比：就像外卖盒子，商家预留了“放主食”“放配菜”“放酱料”的三个插槽，你下单时可以根据喜好往插槽里放不同的内容（比如主食放米饭，配菜放青菜）。

2. 基础原理：从Button的源码看Slots

Button组件的核心源码简化如下，其中 `content` 参数就是最常见的“内容插槽”：

Code block

```
1  // Button组件的简化定义（理解Slots用）
2  @Composable
3  fun Button(
4      onClick: () -> Unit, // 点击事件参数
5      content: @Composable () -> Unit // 内容插槽：接收Composable内容
6  ) {
7      // 按钮的基础样式（背景、圆角等）
8      Box(modifier = /* 按钮样式 */ ) {
9          content() // 执行插槽内容，即使用者传入的Text等组件
10     }
11 }
```

我们调用Button时，就是把Text“插入”到content插槽中：

Code block

```
1  Button(onClick = {}) {
2      // 插入到content插槽的内容
3      Text("点击我")
4  }
```

3. 实战：自定义带多插槽的组件

实际开发中，我们常需要自定义多插槽组件，比如“带标题和内容的卡片”，预留“标题插槽”和“内容插槽”。

Code block

```
1  // 自定义多插槽组件：带标题的卡片
2  @Composable
3  fun TitleCard(
4      // 标题插槽：接收标题内容
5      title: @Composable () -> Unit,
6      // 内容插槽：接收卡片主体内容
7      content: @Composable () -> Unit
8  ) {
9      Card(
```

```

10         modifier = Modifier
11             .fillMaxWidth()
12             .padding(16.dp),
13         elevation = CardDefaults.cardElevation(4.dp)
14     ) {
15         Column(modifier = Modifier.padding(16.dp)) {
16             // 标题插槽内容
17             title()
18             Spacer(modifier = Modifier.height(8.dp))
19             // 内容插槽内容
20             content()
21         }
22     }
23 }
24
25 // 调用自定义插槽组件
26 @Composable
27 fun UseTitleCard() {
28     TitleCard(
29         // 标题插槽：传入标题文本
30         title = { Text("我的订单", fontSize = 18.sp) },
31         // 内容插槽：传入订单详情（多行文本）
32         content = {
33             Text("订单号：20240510001")
34             Text("金额：¥99.00")
35             Text("状态：已支付")
36         }
37     )
38 }
39
40 @Preview(showBackground = true)
41 @Composable
42 fun TitleCardPreview() {
43     UseTitleCard()
44 }

```

优势：通过多插槽，TitleCard组件可以灵活适配不同场景——既可以放订单内容，也可以放用户信息，实现组件的高度复用。

三、使用列表：高效展示“大量数据”

当需要展示“用户列表”“商品列表”等大量数据时，就需要用到Compose的列表组件。Compose提供 `LazyColumn`（垂直列表）和 `LazyRow`（水平列表），它们的核心优势是“懒加载”——只渲染当前可见的列表项，性能远超传统的ListView。

1. LazyColumn：垂直滚动列表（最常用）

功能：垂直方向的滚动列表，类似RecyclerView，是APP中列表展示的首选。

基础用法：静态数据列表

Code block

```
1  import androidx.compose.foundation.lazy.LazyColumn
2  import androidx.compose.foundation.lazy.items
3  import androidx.compose.material3.Card
4  import androidx.compose.material3.Text
5  import androidx.compose.foundation.layout.padding
6  import androidx.compose.ui.unit.dp
7
8  // 数据模型：用户信息
9  data class User(val id: Int, val name: String, val desc: String)
10
11 // 垂直列表组件
12 @Composable
13 fun UserVerticalList(users: List
```

进阶用法：列表项点击与状态更新

给列表项添加点击事件，点击后更新列表项状态（如高亮选中状态）。

Code block

```
1  import androidx.compose.foundation.background
2  import androidx.compose.runtime.mutableStateOf
3  import androidx.compose.runtime.remember
4
5  @Composable
6  fun SelectableUserList(users: List
```

2. LazyRow：水平滚动列表

功能：水平方向的滚动列表，常用于“横向商品展示”“标签栏”等场景，用法和LazyColumn类似，只是排列方向不同。

Code block

```
1  import androidx.compose.foundation.lazy.LazyRow
2
3  // 水平滚动列表：商品标签
4  @Composable
```

四、小结回顾与课后作业

1. 核心知识点速记

知识点	核心作用	关键用法
Column	垂直排列子组件	控制horizontalAlignment（水平对齐）、verticalArrangement（垂直间距）
Row	水平排列子组件	用weight分配水平空间，实现左右对齐布局
Box	层叠对齐子组件	用contentAlignment控制子组件在容器内的位置
Slots API	组件内容灵活组装	用@Composable参数定义插槽，调用时传入子内容
LazyColumn/LazyRow	高效展示列表数据	用items遍历数据，实现懒加载，添加clickable处理点击

2. 课后作业：动手实现“购物车列表”

需求：

- 1. 定义数据模型 `ShoppingCartItem`，包含属性：商品名称（name）、单价（price）、数量（count）、是否选中（isSelected）；
- 2. 用LazyColumn实现购物车列表，每个列表项包含：
复选框（控制isSelected状态）；
- 3. 商品名称和单价（用Column垂直排列）；
- 4. “减少” “+1” 按钮（用Row水平排列），点击可修改count；
- 5. 列表底部用Row实现“全选”按钮和“总价”文本（总价=选中商品的单价×数量之和）；
- 6. 使用自定义Slots组件封装列表项（预留“商品信息”“操作按钮”插槽）。

提示：用mutableStateOf管理列表项的选中状态和数量，总价通过遍历列表计算得出，遇到问题可以参考本节课的列表点击和Slots组件案例。

（注：文档部分内容可能由 AI 生成）